Controlling Transactions and Locks in SQL - Part 2

By Don Schlichting

Introduction

Controlling Transactions and Locks in SQL - Part 1 introduced transactions (the smallest unit of TSQL work). ACID, the acronym that governs transaction integrity, was used as the framework for transaction behavior. In order for a transaction to meet the requirements of ACID, locks are employed to insure data integrity and multi-user access. The scope, or number of rows held by a lock, is referred to as Lock Granularity. This month, we will begin by introducing several different types of lock modes employed by MS SQL.

Lock Modes

SQL utilizes several different types of locks and modes. The way in which a lock shares, or does not share records it is currently working on, are called Lock Modes. This article will focus on the Lock Modes most commonly found in daily SQL work, and the ones we will most likely wish to control during individual transactions.

Exclusive Lock X

This lock mode is very straightforward, a group of records are taken, (row, page, extent, table, or database), and are held exclusively by one transaction. When an Insert, Update, or Delete statement runs, an Exclusive Lock will be issued. No other operation of any kind, (read or DML), can use the records held by an X lock. If the transaction is very long running, such as a nightly update routine, then any reports trying to run while the Exclusive lock is in place will fail. Because of this, SQL will try to release these types of locks as soon as possible.

Shared Lock S

A Shared Lock is applied on records read by a select statement. It is designed to allow concurrent read access from other transactions, but none can modify the held records. This lock enables reads to comply with ACID by disallowing records to be changed at the same instant a read occurs. Only committed data can be read. Shared lock manipulation is one of the areas we will gain performance by controlling.

Deadlocking

Before moving on to the next lock type, a brief review of deadlocking is required. Deadlocking refers to the condition in which one resource is waiting on the action of a second, while that second action is waiting on the first. This is different from being blocked, or having to wait for a resource. Using the locks above, if a transaction had a shared lock, then you issued a delete on those same records held by the first lock, you would not be deadlocked. Instead, you would be blocked. When the shared lock was released, your delete statement would complete. Blocking implies some performance hit, but the transaction will complete. It simply has to wait for something else to finish first. A deadlock on the other hand, means there is no way to finish. Your transaction is stuck in a loop with some other transaction. At this point, the database system will usually pick one transaction to be killed so the other can complete.

Update Lock U

A common type of deadlocking can occur when statements that have Shared locks want to convert them to Exclusive locks. The example usually cited involves one transaction holding a shared lock, while a second transaction also receives a shared lock on the same records, or a subset of records. This causes no problems because more than one transaction can hold a shared lock at the same time on the same data. But now, the first transaction wants to delete some of the rows, so it requests a conversion to an Exclusive lock. The lock cannot be issued because the second transaction still has that shared lock. Therefore, it waits. As the first transaction waits, the second transaction requests an Exclusive lock to also delete or update some records. It cannot be granted because the first transaction still has a shared lock. So now, it also waits. Now, both transactions are waiting on each other to finish so its locks can be converted. Neither will ever finish because they are deadlocked, or stuck in a loop with each other. An Update Lock stops this type of deadlock from occurring. When a transaction signals intent to convert, an Update lock is requested. Only one transaction can obtain an Update lock on the selected resource at a time. When the records are actually modified, the Update lock will be converted to an Exclusive lock. By only allowing one transaction at a time to obtain an Update lock, the deadlocking on conversion requests is eliminated. This type of lock can be understood by looking at an update statement with a WHERE clause. Before the update can complete, the records meeting the WEHRE clause must be selected. Rather than use a shared lock for this initial select, an Update lock will be requested.

Other Locks

SQL employs several other lock types as well, but they will not be used for lock optimization and control in this series. We will briefly introduce them. Schema locks (Sch*), are usually used when a table is being modified, such as column being added. Bulk Update locks (BU), are used for bulk update statements. Intent locks (I*) are used internally by SQL to increase performance.

SP_LOCK

The mode code mentioned after each lock type (such as S for Shared, X for Exclusive), mirror the built in SQL command SP_LOCK. This procedure will return a list of information about locks that have been issued. As an example, we will begin a transaction but not complete it, then run SP_LOCK to view our transaction information. On SQL, run this statement:

```
USE pubs
GO
BEGIN TRAN
 UPDATE jobs
 SET job_desc = 'something'
EXEC sp_lock
```

The following lock information is returned.

	spid	dbid	ObjId	IndId	Type	Resource	Mode	Status
1	51	5	0	0	DB		S	GRANT
2	51	5	277576027	1	KEY	(040004284ada)	X	GRANT
3	51	5	277576027	1	KEY	(09004956e46f)	X	GRANT
4	51	5	277576027	1	PAG	1:115	IX	GRANT
5	51	5	277576027	1	KEY	(08000867ff76)	X	GRANT
6	51	5	277576027	1	KEY	(0500451951c3)	X	GRANT
7	51	5	277576027	0	TAB		IX	GRANT
8	51	5	277576027	1	KEY	(0a008a05c944)	X	GRANT
9	51	5	277576027	1	KEY	(0700c77b67f1)	X	GRANT

Now issue a roll back to end and cancel the transaction:

ROLLBACK

Several of the modes mentioned appear, S for Shared, X for Exclusive. For a complete description of the values returned,
see http://msdn.microsoft.com/library/default.asp?url=/library/en-us/tsqlref/ts_sp_la-lz_6cdn.asp

Conclusion

Lock modes provide SQL with a method of controlling data integrity. With an understanding of the modes, next month we will begin to manipulate them for performance gains.